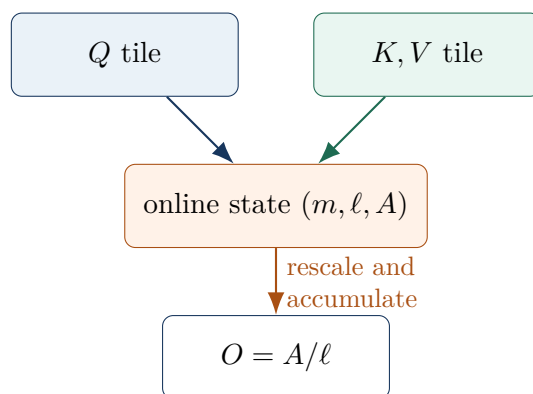


An Educational FlashAttention-Style CUDA Operator

Exact Online Softmax, IO-Aware Tiling, and
First-Order Backpropagation from Scratch



Technical report and reproducible experiment template

FlashAttention CUDA From Scratch Project

Report revision: July 2026

Abstract

Scaled dot-product attention is arithmetically simple but difficult to implement efficiently. A conventional implementation materializes an $N_q \times N_k$ score matrix, normalizes it, and materializes the probability matrix before multiplying by values. Those intermediates have quadratic size and cause repeated traffic through off-chip memory. FLASHATTENTION reorganizes the same exact computation around the memory hierarchy: it tiles keys and values through on-chip storage, evaluates softmax online, and retains only a small state for each query row. This report develops those ideas from first principles and documents an educational PyTorch/CUDA implementation with a complete first-order backward pass.

The operator accepts contiguous CUDA tensors $Q \in \mathbb{R}^{B \times H \times N_q \times D}$ and $K, V \in \mathbb{R}^{B \times H \times N_k \times D}$ in FP16, BF16, or FP32, with $1 \leq D \leq 256$. It supports rectangular non-causal attention and square causal self-attention. The forward kernel launches four warps per thread block, assigns one query row to each warp, and streams key/value tiles of 16 rows through FP32 shared memory. It maintains the running maximum, exponential denominator, and value numerator in FP32. The backward path does not save the probability matrix. Instead, it stores the forward log-sum-exp statistic, computes the rowwise contraction $\delta_i = dO_i \cdot O_i$, and recomputes probabilities in separate query-major and key-major kernels. These kernels write dQ and (dK, dV) without atomics.

The mathematical treatment proves the online-softmax invariant and derives the vector-valued merge rule used by the kernel. It then derives the complete reverse-mode equations, explains why the scalar δ_i is sufficient for the softmax Jacobian, and distinguishes recomputation from approximation. The systems treatment maps those equations onto warps, registers, shared memory, and global memory; calculates the exact shared-memory footprint $128D$ bytes per forward or gradient block; and discusses synchronization, tail handling, memory coalescing, launch geometry, and numerical behavior. The evaluation chapter defines falsifiable correctness and performance protocols, but intentionally reports no GPU claims before a controlled run. Visible placeholders are connected to future generated CSV/JSON artifacts so that absent measurements cannot be mistaken for evidence.

This implementation is not the production FA library. It deliberately excludes dropout, arbitrary masks and biases, grouped-query attention, tensor-core matrix instructions, second-order gradients, and architecture-specific asynchronous pipelines. Its contribution is an inspectable end-to-end realization of the central IO-aware algorithm, including first-order training gradients, together with a disciplined framework for measuring what the implementation actually achieves.

Reader’s guide and evidence policy

The report separates three kinds of statement. A *mathematical claim* follows from stated equations and is accompanied by a derivation or citation. An *implementation claim* describes the source-level contract and launch structure and must be checkable against the repository. An *empirical claim* requires an archived result artifact produced on named hardware and software. The first two kinds are developed now. The third remains explicitly pending until the project is executed on a CUDA-capable Kaggle session or another recorded machine.

Chapters 2–4 develop attention, the GPU memory hierarchy, online softmax, and tiled exact attention. Chapters 5–7 specify the native interface and forward/backward kernels. Chapters 8–10 define validation, experimental methodology, and data-driven result hooks. The remaining chapters interpret what the eventual evidence can and cannot establish. Appendices provide expanded derivations, a reproducibility checklist, and machine-readable result schemas.

Scope boundary

The term *FlashAttention-style* in this report means an exact attention computation that avoids materializing the quadratic score/probability matrices by combining tiling with online normalization and backward recomputation. It does not imply API, feature, or performance parity with the production FlashAttention releases of Dao et al. [2022], Dao [2024], Shah et al. [2024].

Table 1: Principal symbols used throughout the report. Batch and head indices are usually suppressed after flattening them into one independent problem.

Symbol	Shape or domain	Meaning
B, H	positive integers	batch size and number of attention heads
N_q, N_k	positive integers	query and key/value sequence lengths
D	$1 \leq D \leq 256$	query/key head dimension and value dimension in this implementation
Q, K, V	$N_q \times D, N_k \times D, N_k \times D$	one batch-head slice of the input tensors
α	positive scalar	score scale; default convention is $D^{-1/2}$
S	$N_q \times N_k$	scaled and, if causal, masked scores
P	$N_q \times N_k$	rowwise softmax probabilities
O	$N_q \times D$	attention output PV
m_i, ℓ_i	scalars per query row	online maximum and rescaled exponential sum
A_i	$\mathbb{R}^D$	unnormalized online value numerator
L_i	scalar per query row	log-sum-exp $m_i + \log \ell_i$ saved for backward
G or dO	$N_q \times D$	upstream derivative of the scalar loss with respect to O
δ_i	scalar per query row	contraction $dO_i \cdot O_i$ used in the softmax gradient
M	element capacity	abstract fast-memory capacity in IO-complexity analysis
B_c	16	key/value or query tile length in the native kernels

Contents

Abstract	1
Reader’s guide and evidence policy	2
1 Introduction	10
1.1 Why attention is a systems problem	10
1.2 Project objective	11
1.3 Implemented contract	11
1.4 Contributions of the report	13
1.5 Research questions	13
2 Background: Attention and the GPU Memory Hierarchy	15
2.1 Scaled dot-product attention	15
2.2 Materialized evaluation	16
2.3 A two-level view of GPU memory	16
2.4 Arithmetic intensity and fusion	17
2.5 Exact memory efficiency is distinct from approximate attention	18
3 Online Softmax as a Streaming Invariant	19
3.1 Why softmax appears to resist tiling	19
3.2 Scalar online normalization	19
3.3 Vector-valued sufficient state	20
3.4 Tile merges and exactness	21
3.5 Finite-precision behavior	21
3.6 Implementation consequence for the forward kernel	21
4 FlashAttention as an IO-Aware Schedule	22
4.1 Two notions of efficiency	22
4.2 The two-level memory model	22
4.3 Algorithm sketch	23
4.4 Why the schedule matters on GPUs	23
4.5 From the abstract schedule to this repository	24
4.6 Exactness, recomputation, and training	24
4.7 What the method does not claim	24

5	CUDA Kernel Design	25
5.1	Mapping one attention problem to the grid	25
5.2	Per-lane data ownership	25
5.3	Forward shared-memory staging	26
5.4	Warp reductions and scalar state	26
5.5	Causal control flow	26
5.6	Backward launch partition	27
5.7	Occupancy and resource trade-offs	27
5.8	Host boundary, streams, and safety	27
5.9	Implication for profiling	28
6	Repository Implementation and Software Interface	29
6.1	Why software structure matters	29
6.2	Layered repository design	29
6.3	Public Python API	29
6.4	Reference implementations	30
6.5	Native binding contract	30
6.6	Packaging and Kaggle build behavior	31
6.7	Benchmark and artifact pipeline	31
6.8	Local continuity files	31
6.9	Why the interface is part of the technical contribution	31
7	Backward Pass by Recomputation	33
7.1	Why the backward pass is the hard part	33
7.2	Reverse-mode equations	33
7.3	Reconstructing probabilities	34
7.4	Delta prepass	34
7.5	Query-major kernel for dQ	34
7.6	Key-major kernel for dK and dV	34
7.7	Causal handling in backward	35
7.8	Cost model of recomputation	35
7.9	Numerical considerations	35
7.10	Why the backward design is portfolio-worthy	36
8	Correctness and Validation Strategy	37
8.1	Validation goals	37
8.2	Reference choice	37
8.3	Test layers	37
8.4	Shape and dtype coverage	37
8.5	Tolerance policy	38
8.6	Non-contiguous and stream-sensitive behavior	38
8.7	What the current suite does not yet automate	38
8.8	Strict publication gate	39

8.9	Interpreting a passing result	39
9	Experimental Methodology	40
9.1	Principles	40
9.2	Measured implementations	40
9.3	Shape matrix	40
9.4	Timing procedure	41
9.5	Reported metrics	41
9.6	Environment capture	41
9.7	Kaggle protocol	42
9.8	From benchmark artifact to report	42
9.9	What would constitute convincing evidence	42
10	Results, Placeholders, and Future Evidence	44
10.1	Status of empirical evidence	44
10.2	Environment summary	44
10.3	Correctness summary	44
10.4	Performance comparison	45
10.5	Memory behavior	45
10.6	Profiler and sanitizer evidence	45
10.7	Publication rules for future updates	45
11	Discussion	47
11.1	What this project demonstrates already	47
11.2	Why an educational kernel may still lose to production code	47
11.3	What a positive result would mean	47
11.4	What a negative result would still teach	48
11.5	Relationship to the broader literature	48
11.6	Portfolio perspective	48
12	Limitations and Honest Boundaries	49
12.1	Native feature limits	49
12.2	Validation limits	49
12.3	Performance interpretation limits	50
12.4	Scientific scope	50
12.5	Future work	50
13	Conclusion	51
A	Additional Derivations	52
A.1	Merging two processed key sets	52
A.2	Softmax Jacobian contraction	52
A.3	Causal work count	53
A.4	Shared-memory footprint	53

A.5	Dense-equivalent FLOP model	53
B	Reproducibility Checklist	54
B.1	Kaggle execution checklist	54
B.2	Suggested artifact bundle	54
B.3	Recommended sanitizer and profiler commands	55
B.4	Worktree and provenance policy	55
B.5	Report update policy	55
C	Benchmark and Report Artifact Schema	56
C.1	Benchmark JSON structure	56
C.2	Representative row fields	56
C.3	Rendered report snippets	57
C.4	Expected workflow	57
C.5	Why a schema appendix belongs in the report	58

List of Figures

2.1	Logical dataflow of materialized attention. The drawing denotes data dependence, not a claim that a particular PyTorch backend necessarily creates both intermediates.	16
2.2	Original schematic of the memory scopes relevant to this implementation. Physical sizes and bandwidths depend on the GPU and must be captured from the actual experiment rather than assumed from a paper’s device.	17

List of Tables

1	Principal symbols used throughout the report. Batch and head indices are usually suppressed after flattening them into one independent problem.	3
1.1	Implemented feature contract. “Not supported” is a deliberate boundary, not an unmeasured claim.	12
4.1	Qualitative comparison between three attention evaluation strategies.	23
5.1	Static design parameters of the native CUDA kernels.	25
8.1	Automated validation layers currently implemented in the repository.	38
9.1	Primary benchmark outputs written per successful row.	41
12.1	Current limitations of the native implementation and why they matter.	49
C.1	Representative fields in one benchmark result row.	56

Chapter 1

Introduction

1.1 Why attention is a systems problem

The Transformer made scaled dot-product attention a standard computational primitive rather than a specialized modeling technique [Vaswani et al., 2017]. For one attention head, the mathematical definition occupies a single line:

$$O = \text{softmax}\left(\alpha QK^T + M_{\text{mask}}\right)V. \quad (1.1)$$

Here $Q \in \mathbb{R}^{N_q \times D}$, $K, V \in \mathbb{R}^{N_k \times D}$, and typically $\alpha = D^{-1/2}$. Equation (1.1) conceals an implementation choice. One may first write all $N_q N_k$ scores to device memory, read them to compute softmax, write all $N_q N_k$ probabilities, and read them again for the value product. Alternatively, one may change the evaluation order so that only small score blocks exist on chip. Both paths perform the same mathematical function. Their memory traffic and attainable wall time can be very different.

This distinction matters because a GPU is not a uniform arithmetic engine. It is a hierarchy containing a large but relatively distant global-memory space, caches, explicitly managed shared memory, and a small register file associated with each thread. Threads execute in warps and thread blocks; data reuse is effective only when the algorithm and mapping expose it at the appropriate scope. Modern accelerators can issue arithmetic faster than many workloads can feed operands from off-chip memory. Consequently, counting floating-point operations alone does not predict performance. The Roofline model makes this dependence explicit through arithmetic intensity, the ratio of useful work to bytes moved [Williams et al., 2009]. Transformer-level studies likewise find that data movement is often a decisive optimization target [Ivanov et al., 2021].

FLASHATTENTION treats attention as an IO-aware algorithm [Dao et al., 2022]. Its central mechanism is not a new approximation to softmax. It is a new schedule for exact attention. Key and value blocks are loaded into fast memory, query rows consume them, and an online normalizer updates the output without requiring the complete row of scores at once. During backward propagation, probabilities can be reconstructed from Q , K , and a log-normalization statistic instead of being retained as a quadratic activation. The original paper establishes reduced HBM access under an explicit two-level memory model and shows that additional computation can be profitable when it avoids more expensive traffic.

The result is a useful case study in the relationship among algebra, numerical analysis, and parallel programming. A correct implementation must answer all of the following questions:

- Which compact state is sufficient to merge independently computed score blocks without changing the normalized result?

- How must an existing output numerator be rescaled when a later block contains a larger maximum?
- Which forward statistics make the softmax probabilities recoverable during reverse mode?
- How can gradients for queries, keys, and values be partitioned so that each output element has one writer?
- Where do query, key, value, probability, and gradient fragments reside in the GPU hierarchy at each point?
- Which errors are expected from low-precision inputs and reassociated sums, and which errors indicate a defect?
- How can a benchmark distinguish kernel execution, dispatch overhead, memory allocation, and an optimized framework baseline that may itself select FlashAttention?

This report answers those questions for a deliberately compact native CUDA implementation integrated with PyTorch.

1.2 Project objective

The objective is to implement and explain an exact, differentiable FlashAttention-style operator from first principles. “From first principles” does not mean ignoring established work: the attention definition, online-normalizer recurrence, IO-aware schedule, and GPU programming model are cited to their primary or official sources. It means that the project exposes its own forward and backward kernels instead of calling a pre-existing fused attention operator. The result should be small enough that a reader can trace one query row from Python dispatch through CUDA execution and back through the gradient formulas.

The educational objective imposes different priorities from a production library. Production implementations specialize aggressively across architectures, head dimensions, dtypes, masking modes, and workload shapes; use tensor-core matrix instructions; pipeline asynchronous memory movement; and provide mature integration with broader model stacks [Dao, 2024, Shah et al., 2024, Dao and contributors, 2026]. This project instead uses scalar CUDA arithmetic, warp reductions, explicit FP32 shared-memory tiles, and a fixed four-warp launch structure. That design leaves performance on the table, but makes the online state, work ownership, and synchronization visible.

1.3 Implemented contract

The native extension exposes a forward operation

$$\text{forward}(q, k, v, \text{causal}, \text{scale}) \longrightarrow (o, L)$$

and a backward operation

$$\text{backward}(do, q, k, v, o, L, \text{causal}, \text{scale}) \longrightarrow (dq, dk, dv).$$

The public autograd facade connects these operations into a first-order differentiable PyTorch function. A legacy three-input forward-only entry point remains available for compatibility, but it does not define the full supported interface.

The constrained contract is still broad enough to expose the difficult cases. Sequence lengths and D need not be multiples of the warp width or tile length. Rectangular non-causal attention exercises distinct query and key loop bounds. Causal attention changes the valid domain per query row. Low-precision inputs require careful accumulation. The backward pass must combine a query-major computation for dQ with a key-major computation for dK and dV while recomputing probabilities consistently.

Table 1.1: Implemented feature contract. “Not supported” is a deliberate boundary, not an unmeasured claim.

Capability	Status	Exact condition
Tensor layout	Supported	contiguous CUDA tensors in $[B, H, N, D]$ order; native code rejects non-contiguous inputs rather than silently copying
Input dtype	Supported	Q, K, and V share FP16, BF16, or FP32 dtype
Head dimension	Supported	$1 \leq D \leq 256$ and the value dimension equals D
Non-causal attention	Supported	N_q and N_k may differ; batch, head, and head dimension must match
Causal attention	Supported	square self-attention only, $N_q = N_k$; positions $j > i$ are excluded
Forward normalization	Supported	stable online softmax with FP32 state and FP32 saved log-sum-exp
First-order gradients	Supported	gradients with respect to Q, K, and V through a custom backward
Dropout	Not supported	no random keep mask and no dropout scaling
Arbitrary masks/biases	Not supported	no padding mask, additive bias, relative bias, or ALiBi
Grouped/multi-query attention	Not supported	Q, K, and V have the same number of heads
Second-order gradients	Not supported	backward is a terminal native operation, not a differentiable graph of PyTorch primitives
Tensor cores	Not used	dot products are scalar/SIMT reductions; no WMMA/MMA path

1.4 Contributions of the report

This document makes six concrete contributions to the repository.

First, it supplies a self-contained derivation of online softmax. The scalar normalizer recurrence is lifted to a vector-valued sufficient statistic whose numerator directly accumulates the weighted values. The merge operation is proved correct for arbitrary partitions, not only for the fixed tile size used by the code.

Second, it states the distinction between arithmetic complexity, activation memory, and HBM traffic. FlashAttention-style scheduling does not turn dense exact attention into a linear-time algorithm. It performs $\Theta(N_q N_k D)$ arithmetic while avoiding the $\Theta(N_q N_k)$ score and probability allocations. Under the memory model of Dao et al. [2022], tiling reduces the asymptotic number of slow-memory accesses over the relevant fast-memory range.

Third, it documents the native execution map precisely: 128 threads, four warps, one output row per warp, a key/value tile of 16, and $128D$ bytes of dynamic FP32 shared memory. It derives index ownership and identifies every block-wide synchronization point. The same reasoning is applied to the query-major and key-major gradient kernels.

Fourth, it derives full first-order reverse mode. The derivation shows why $\delta_i = dO_i \cdot O_i$ equals the probability-weighted sum required by the softmax Jacobian and why storing $L_i = \log \sum_j e^{S_{ij}}$ is sufficient to reconstruct P_{ij} .

Fifth, it defines a rigorous validation protocol. The protocol covers reference construction, forced PyTorch SDPA backends, finite-difference spot checks, dtype-specific tolerances, extreme logits, non-multiple tails, rectangular shapes, causal boundaries, invalid inputs, and CUDA memory/race/synchronization checks.

Sixth, it creates an honest empirical template. Results enter through generated files tied to CSV/JSON provenance. A missing artifact produces an orange placeholder, never a fabricated number or an empty plot that might be mistaken for a successful run.

1.5 Research questions

The eventual experiments are organized around four research questions rather than a search for a favorable headline number.

RQ1: Numerical agreement.

Across supported shapes, dtypes, and causal modes, do forward outputs and first-order gradients agree with a high-confidence PyTorch reference within declared tolerances?

How does error change with sequence length, head dimension, and logit magnitude?

RQ2: Memory behavior.

Does the implementation avoid allocating an $N_q N_k$ score or probability tensor, and does measured peak allocated memory grow consistently with linear saved state for fixed B, H, D ?

RQ3: Runtime scaling.

At which shapes, if any, does fusion and reduced traffic compensate for scalar arithmetic and launch overhead relative to a manual materialized baseline? How far does the educational kernel remain from PyTorch’s optimized fused backend?

RQ4: Hardware explanation.

Do profiler counters support the proposed explanation of runtime—for example, memory traffic saved but limited instruction throughput, occupancy, or reduction efficiency? Which resource becomes limiting as D grows?

These questions allow an unimpressive speed result to remain scientifically useful. A kernel

can correctly demonstrate the algorithm while losing to production code. Conversely, a fast number without numerical checks, backend control, or profiler evidence would not establish the intended result.

Chapter 2

Background: Attention and the GPU Memory Hierarchy

2.1 Scaled dot-product attention

For a single batch element and attention head, let the i th query, j th key, and j th value be row vectors $q_i, k_j, v_j \in \mathbb{R}^D$. The unmasked score is

$$s_{ij} = \alpha q_i k_j^\top = \alpha \sum_{d=0}^{D-1} q_{id} k_{jd}. \quad (2.1)$$

For a valid-index set $\mathcal{V}_i \subseteq \{0, \dots, N_k - 1\}$, rowwise probabilities and output are

$$p_{ij} = \begin{cases} \frac{e^{s_{ij}}}{\sum_{t \in \mathcal{V}_i} e^{s_{it}}}, & j \in \mathcal{V}_i, \\ 0, & j \notin \mathcal{V}_i, \end{cases} \quad (2.2)$$

$$o_i = \sum_{j \in \mathcal{V}_i} p_{ij} v_j. \quad (2.3)$$

Non-causal attention uses $\mathcal{V}_i = \{0, \dots, N_k - 1\}$ and therefore permits $N_q \neq N_k$. Square causal self-attention uses $N_q = N_k = N$ and $\mathcal{V}_i = \{0, \dots, i\}$. This report treats masking as a change in the summation domain. The equivalent dense convention assigns $s_{ij} = -\infty$ outside $\mathcal{V}_i$.

The usual scale $\alpha = D^{-1/2}$ controls score magnitude. Suppose, only for this variance argument, that components of q_i and k_j are independent, zero mean, and unit variance. The unscaled dot product is a sum of D zero-mean products, each with unit variance, so

$$\text{Var} \left(\sum_{d=0}^{D-1} q_{id} k_{jd} \right) = D. \quad (2.4)$$

Multiplication by $D^{-1/2}$ returns the variance to one. Without scaling, increasing D tends to push softmax farther into saturation, where one entry dominates and derivatives through most entries become small. This argument motivates a default, but the native API accepts an explicit positive scale so a caller can reproduce another attention convention.

With batch and heads restored, each (b, h) slice is independent:

$$Q, K, V \in \mathbb{R}^{B \times H \times N \times D}. \quad (2.5)$$

The implementation exploits that independence in the grid geometry. It does not implement grouped-query attention, in which query heads outnumber key/value heads and a key/value slice is shared across several query heads.

2.2 Materialized evaluation

A conventional decomposition contains three logical stages:

$$S = \alpha QK^\top, \quad P = \text{softmax}(S), \quad O = PV. \quad (2.6)$$

Calling optimized matrix multiplication for the first and third stages can provide excellent compute throughput. The decomposition nevertheless creates a systems cost: S and P each contain $N_q N_k$ elements per batch-head pair. If both are materialized in an element type of w bytes, their payload alone is

$$\text{bytes}(S, P) = 2wBHN_q N_k. \quad (2.7)$$

At $B = 2$, $H = 16$, $N_q = N_k = 4096$, and two-byte storage, this expression is 2 GiB, before Q/K/V/O, allocator overhead, workspace, gradients, or other model activations. An actual framework may fuse or avoid some intermediates, so (2.7) describes the explicit materialized algorithm rather than every call to a high-level API.

The leading arithmetic count is also quadratic. Treating a fused multiply-add as two floating-point operations, the score product and value product each cost approximately $2N_q N_k D$ operations per head, giving

$$F_{\text{fwd}} \approx 4BHN_q N_k D \quad (2.8)$$

plus lower-order exponentials, reductions, masking, and rescaling. Causal attention visits approximately half the square score domain, although boundary and diagonal tiles prevent an exact factor of two for finite N . Tiled exact attention changes neither the dense non-causal asymptotic arithmetic order nor the mathematical output.

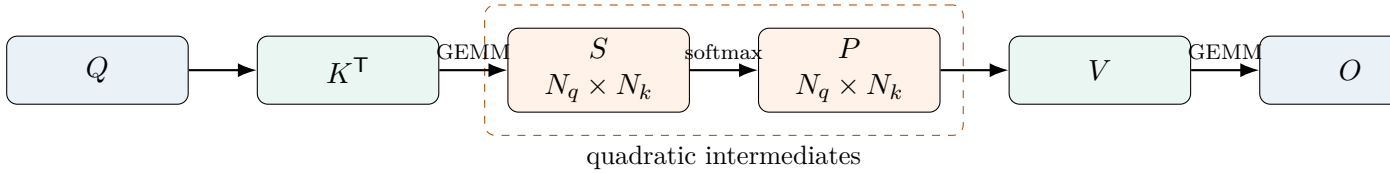


Figure 2.1: Logical dataflow of materialized attention. The drawing denotes data dependence, not a claim that a particular PyTorch backend necessarily creates both intermediates.

2.3 A two-level view of GPU memory

IO-aware analysis abstracts a complex GPU into a large slow memory and a small fast memory. In the terminology used by Dao et al. [2022], HBM is the large off-chip level and SRAM is the small on-chip level. Actual NVIDIA hardware has more levels and scopes: device global memory, L2, an SM-local L1/shared-memory complex, and per-thread registers. The CUDA programming model exposes thread blocks that cooperate through shared memory and barriers, while threads within a warp execute in SIMT groups and can exchange values with shuffle instructions [NVI, 2026c, Nickolls et al., 2008].

The word “fast” is contextual. Registers avoid explicit shared-memory instructions, but excessive per-thread register demand can reduce the number of resident warps or spill into local memory, which resides off chip. Shared memory offers low-latency reuse within a block, but bank conflicts and barriers can serialize progress. Global accesses can achieve high bandwidth when a warp addresses contiguous segments, yet inefficient access patterns transfer unused

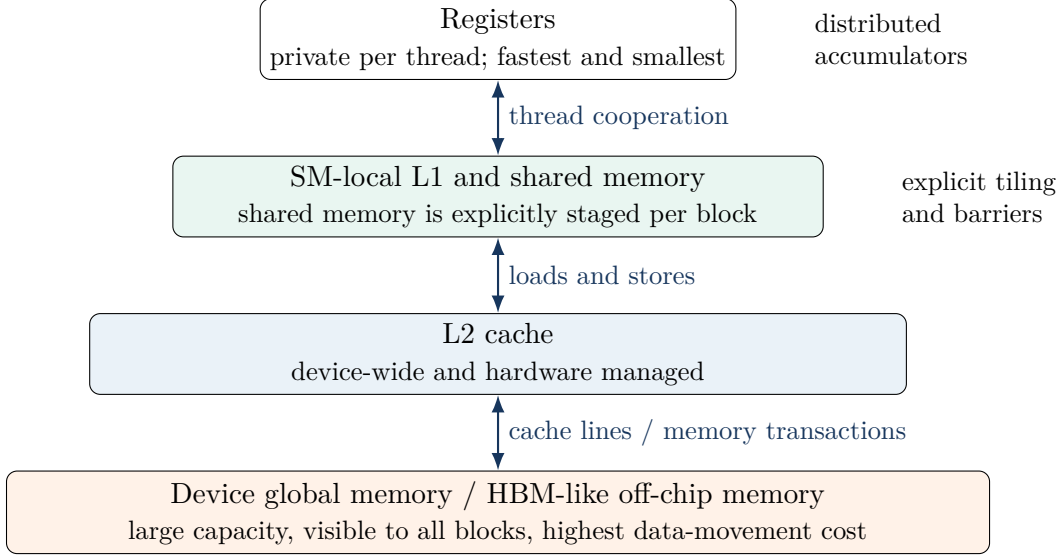


Figure 2.2: Original schematic of the memory scopes relevant to this implementation. Physical sizes and bandwidths depend on the GPU and must be captured from the actual experiment rather than assumed from a paper’s device.

bytes. NVIDIA therefore recommends coalescing global accesses and using shared memory for data reuse when the reuse offsets staging and synchronization costs [NVI, 2026a].

The implementation’s fixed mapping creates all three trade-offs. A warp distributes the D components of one query and one output accumulator across 32 lanes. Larger D increases registers per thread because each lane owns several output elements. Key and value tiles are converted to FP32 in shared memory, so the per-block footprint grows linearly with D . Four query rows share each staged key/value tile, providing reuse but also requiring every warp to reach the same block-wide barriers, even when a tail query row is inactive.

2.4 Arithmetic intensity and fusion

For a kernel with F relevant operations and T bytes transferred at a chosen memory level, arithmetic intensity is $I = F/T$. A simplified Roofline bound is

$$\text{performance} \leq \min(P_{\max}, I\beta_{\max}), \quad (2.9)$$

where $P_{\max}$ is peak arithmetic throughput and $\beta_{\max}$ is peak bandwidth at that level [Williams et al., 2009]. The bound is diagnostic, not a promise that either ceiling will be reached. Instruction mix, dependencies, occupancy, reductions, divergence, and launch overhead can leave a kernel below both ceilings.

Fusion improves intensity when an intermediate can remain on chip instead of being written and reread. Attention is a particularly strong opportunity because the score and probability matrices are much larger than the rowwise statistics needed to normalize them. The obstacle is softmax: every probability in a row depends on the denominator over the complete row. The next chapter shows that this global dependency can be summarized by a maximum and rescaled sum, enabling exact blockwise evaluation.

2.5 Exact memory efficiency is distinct from approximate attention

Many efficient-attention methods change the mathematical problem using sparsity, low-rank structure, kernels, or restricted receptive fields. Such methods can reduce arithmetic complexity, but may change model behavior. Memory-efficient exact attention takes a different route. [Rabe and Staats \[2021\]](#) demonstrate that quadratic storage is not mathematically necessary even while time remains quadratic. FLASHATTENTION adds explicit IO analysis and a GPU schedule that exploits tiling and recomputation [[Dao et al., 2022](#)].

The distinction prevents two common misstatements. First, exact FlashAttention is not “linear-time attention.” Dense non-causal forward work remains $\Theta(N_q N_k D)$. Second, “linear memory” refers to avoiding quadratic intermediates; it does not mean constant total memory. Q, K, V, O, the saved log-normalizers, gradients, and temporary row statistics all scale with sequence length. This report reports each category separately rather than using one undifferentiated memory-complexity label.

Chapter 3

Online Softmax as a Streaming Invariant

3.1 Why softmax appears to resist tiling

Matrix multiplication tiles naturally because partial products may be summed in any associative grouping. Softmax is less forgiving. The normalized probability

$$p_{ij} = \frac{e^{s_{ij}}}{\sum_{t \in \mathcal{V}_i} e^{s_{it}}} \quad (3.1)$$

depends on a denominator that is global to row i . A blockwise implementation that evaluates only one score tile at a time must therefore answer a precise question: what is the smallest state that can summarize the already processed keys so that a later tile can be merged exactly?

The usual stable-softmax identity already suggests the answer. For any finite row vector x ,

$$\text{softmax}(x)_j = \frac{e^{x_j - m}}{\sum_t e^{x_t - m}}, \quad m = \max_t x_t. \quad (3.2)$$

Subtracting m does not change the normalized result, but it prevents positive overflow in the exponentials. The obstacle is that a streaming kernel does not know the final row maximum when it starts on the first tile. The maximum itself must therefore become part of the carried state.

3.2 Scalar online normalization

Milakov and Gimelshein [2018] show that the normalization for a softmax row can be updated online. Suppose a query row has processed a valid prefix J of keys and define

$$m_J = \max_{j \in J} s_{ij}, \quad \ell_J = \sum_{j \in J} e^{s_{ij} - m_J}. \quad (3.3)$$

When a new valid score x arrives, the updated statistics are

$$m' = \max(m_J, x), \quad (3.4)$$

$$\ell' = e^{m_J - m'} \ell_J + e^{x - m'}. \quad (3.5)$$

The first term rescales the old denominator from the old reference maximum m_J to the new reference maximum m' . The second term adds the new contribution under the same reference.

This recurrence is exact in real arithmetic and numerically stable in floating point because both exponentials have non-positive arguments.

The custom CUDA forward kernel uses the recurrence one valid key row at a time rather than one tile at a time. That choice avoids an inner tile reduction for the maximum and denominator. It also makes the causal case straightforward: invalid future keys are skipped, so they never enter the recurrence as $-\infty$ placeholders.

3.3 Vector-valued sufficient state

Attention requires more than the scalar normalizer. The output row

$$o_i = \sum_{j \in \mathcal{V}_i} p_{ij} v_j \quad (3.6)$$

may be rewritten as a ratio between a vector numerator and the scalar denominator. Define

$$A_J = \sum_{j \in J} e^{s_{ij} - m_J} v_j \in \mathbb{R}^D. \quad (3.7)$$

The triplet (m_J, ℓ_J, A_J) is sufficient to recover the exact partial attention result for the processed keys:

$$o_i^{(J)} = \frac{A_J}{\ell_J}. \quad (3.8)$$

If a new valid score-value pair (x, v) arrives, the update becomes

$$m' = \max(m_J, x), \quad (3.9)$$

$$\ell' = e^{m_J - m'} \ell_J + e^{x - m'}, \quad (3.10)$$

$$A' = e^{m_J - m'} A_J + e^{x - m'} v. \quad (3.11)$$

The scalar and vector updates share the same rescaling factor. In implementation terms, each lane of a warp owns a slice of A_J , while lane zero owns the scalar (m_J, ℓ_J) and broadcasts the rescaling coefficients to the rest of the warp.

Invariant 3.1 (Streaming attention state). For any non-empty processed key set $J \subseteq \mathcal{V}_i$, let

$$m_J = \max_{j \in J} s_{ij}, \quad \ell_J = \sum_{j \in J} e^{s_{ij} - m_J}, \quad A_J = \sum_{j \in J} e^{s_{ij} - m_J} v_j.$$

After any sequence of updates of the form (3.11), the maintained state equals (m_J, ℓ_J, A_J) for the set of keys processed so far.

Proof. The empty state is represented by $m = -\infty$, $\ell = 0$, and $A = 0$. The first valid element is treated as a special case: the implementation writes $m = x$, $\ell = 1$, and $A = v$. This equals the invariant for $J = \{j\}$.

Assume the invariant holds for a processed set J and a new valid key $u \notin J$ with score $x = s_{iu}$ is added. Let $m' = \max(m_J, x)$. For every old element $j \in J$,

$$e^{s_{ij} - m'} = e^{s_{ij} - m_J} e^{m_J - m'}. \quad (3.12)$$

Multiplying the old sums by $e^{m_J - m'}$ therefore changes only the common reference maximum, not the represented values. Adding the new contribution under the same reference yields

$$\ell' = \sum_{j \in J} e^{s_{ij} - m'} + e^{x - m'} = \sum_{j \in J \cup \{u\}} e^{s_{ij} - m'}, \quad (3.13)$$

$$A' = \sum_{j \in J} e^{s_{ij} - m'} v_j + e^{x - m'} v_u = \sum_{j \in J \cup \{u\}} e^{s_{ij} - m'} v_j. \quad (3.14)$$

Hence the invariant holds for $J \cup \{u\}$. Induction proves the claim. $\square$

3.4 Tile merges and exactness

The proof above is agnostic to how the key set is partitioned. One may process scores one at a time, tile by tile, or merge independently reduced subproblems so long as each merge applies the same rescaling logic. This is what makes the algorithm both exact and parallelizable. The tile size affects only the schedule and hardware behavior, not the mathematical output in real arithmetic.

This distinction is important because the word “online” sometimes suggests approximation. That is not the case here. The result differs from materialized attention only through floating-point effects that are unavoidable whenever operations are regrouped. [Goldberg \[1991\]](#) and [Higham \[2002\]](#) make the general point: floating-point arithmetic is not associative, so any parallel reduction scheme may change low-order bits even when it computes the same real-valued expression.

3.5 Finite-precision behavior

The implementation uses FP32 for the online state even when the input and output tensors are FP16 or BF16. This is a deliberate numerical design choice.

1. The maximum m_i should be represented in a format with enough exponent range to avoid turning a valid rescaling factor into zero or infinity unnecessarily.
2. The denominator ℓ_i accumulates many positive terms. FP32 reduces the risk that repeated additions of small terms into a large running sum disappear too early.
3. The numerator A_i inherits all the conditioning issues of both the probabilities and the values. Accumulating it in FP16 would introduce rounding at every tile update rather than only at the final store.

The result is still not bitwise identical to every framework backend. Different reduction orders, different handling of fused operations, and hardware-specific implementations of ‘expf’ can change the last bits [[IEE, 2019](#), [PyT, 2026c](#)]. For this reason the test suite uses justified tolerances rather than exact equality for low-precision CUDA cases.

3.6 Implementation consequence for the forward kernel

The invariant drives the structure of the forward kernel directly:

1. each warp loads one query row into registers;
2. the block stages a key tile and a value tile in shared memory;
3. the warp computes one score at a time for the valid rows in the tile;
4. lane zero updates (m_i, ℓ_i) and broadcasts the rescaling factors;
5. every lane rescales its slice of A_i and adds the weighted value slice;
6. after the final tile, the warp writes $o_i = A_i/\ell_i$ and $L_i = m_i + \log \ell_i$.

The saved statistic L_i is not merely an implementation convenience. It is the row log-sum-exp,

$$L_i = \log \sum_{j \in \mathcal{V}_i} e^{s_{ij}}, \quad (3.15)$$

and therefore the exact scalar needed to reconstruct probabilities during the backward pass:

$$p_{ij} = e^{s_{ij} - L_i}. \quad (3.16)$$

The next chapter uses this fact to connect the invariant to the larger IO-aware FlashAttention schedule of [Dao et al. \[2022\]](#).

Chapter 4

FlashAttention as an IO-Aware Schedule

4.1 Two notions of efficiency

Attention kernels are often described as “efficient” without specifying which resource is being reduced. For dense exact attention there are at least three distinct questions:

1. How many arithmetic operations are performed?
2. How much auxiliary memory is allocated?
3. How many bytes are transferred through the expensive levels of the memory hierarchy?

Materialized attention and FlashAttention-style attention have the same leading arithmetic order, $\Theta(N_q N_k D)$, for dense non-causal problems. What changes is the second and third item. The tiled algorithm avoids storing the full score and probability matrices and instead revisits inputs while keeping only compact row statistics and on-chip tiles [Dao et al., 2022, Rabe and Staats, 2021].

4.2 The two-level memory model

Dao et al. [2022] formalize this optimization using a two-level model inspired by classic IO-complexity analysis [Aggarwal and Vitter, 1988]. A large slow memory holds the inputs and outputs, while a smaller fast memory of capacity M elements holds blocks that are actively reused. On an NVIDIA GPU the physical hierarchy is richer than this abstraction, but the model still captures the dominant design decision: it is cheaper to reuse data that has already been staged on chip than to materialize and revisit a large intermediate in HBM.

For attention, the materialized route effectively performs

$$(Q, K) \rightarrow S, \quad S \rightarrow P, \quad (P, V) \rightarrow O. \quad (4.1)$$

Even if an implementation fuses some adjacent steps, a score-like or probability-like object with $N_q N_k$ elements is the natural intermediate. FlashAttention instead transforms the schedule into a sequence of tile updates:

$$(Q_i, K_{t:t+B_c}, V_{t:t+B_c}, m_i, \ell_i, A_i) \longrightarrow (m'_i, \ell'_i, A'_i). \quad (4.2)$$

The full score matrix is never committed to global memory.

4.3 Algorithm sketch

At the algorithmic level, exact tiled attention can be summarized as follows.

1. Partition the key and value matrices into row tiles of height B_c .
2. Partition the query matrix into row tiles of height B_r .
3. For one query tile, initialize online state (m_i, ℓ_i, A_i) for each query row.
4. For every key/value tile:
 - (a) load the tile into fast memory;
 - (b) compute local query-key scores for the tile;
 - (c) update the online statistics and value numerators;
 - (d) discard the score tile after use.
5. Normalize each output row at the end.

This algorithm is exact because the online state is a sufficient statistic for the processed keys. It is memory efficient because the transient object has size $B_r B_c$ rather than $N_q N_k$.

Table 4.1: Qualitative comparison between three attention evaluation strategies.

Strategy	Arithmetic order	Quadratic intermediate	Main trade-off
Materialized exact attention	$\Theta(N_q N_k D)$	yes	simplest conceptual decomposition but heavy memory traffic
FlashAttention-style exact attention	$\Theta(N_q N_k D)$	no	more control flow and re-computation in exchange for lower IO
Approximate efficient attention	varies by method	often no	changes the mathematical operator to reduce work or storage

4.4 Why the schedule matters on GPUs

The GPU is a bandwidth- and latency-sensitive machine. Registers and shared memory provide fast reuse but have small capacity. Global memory is large but expensive to traverse repeatedly. The Roofline model summarizes the tension:

$$\text{attainable throughput} \leq \min(P_{\max}, I\beta_{\max}), \quad (4.3)$$

where I is arithmetic intensity and $\beta_{\max}$ is peak bandwidth at the chosen memory level [Williams et al., 2009]. If an algorithm writes and rereads a huge intermediate, its arithmetic intensity at the HBM boundary may be far below what the arithmetic units could otherwise sustain.

FlashAttention increases effective intensity by fusing multiple logical stages around the on-chip tile. The score exists briefly inside the kernel, contributes immediately to the denominator and value numerator, and is then discarded. This is why the method is best interpreted as a schedule transformation rather than a new algebraic layer.

4.5 From the abstract schedule to this repository

The custom kernel in this repository fixes $B_r = 4$ and $B_c = 16$. Those values are not claimed to be optimal. They are chosen because they fit naturally into one 128-thread block with four warps and because they keep shared-memory usage easy to analyze:

$$\text{shared bytes per block} = 2 \times B_c \times D \times \text{sizeof(float)} = 128D. \quad (4.4)$$

At the maximum supported head dimension $D = 256$, the block uses 32 KiB of dynamic shared memory, which fits under a common 48 KiB default limit.

The fixed tiling gives the project a concrete pedagogical advantage. One may directly connect each theoretical quantity to a code location:

- m_i and ℓ_i are lane-zero scalars in a warp;
- A_i is distributed across warp lanes as per-dimension register slices;
- K and V tiles reside in FP32 shared memory;
- the output row and saved log-sum-exp are written once after all tiles are consumed.

4.6 Exactness, recomputation, and training

The IO-aware perspective continues into backward propagation. A naive autograd decomposition would save probabilities or scores so they are available to the gradient kernels. That saves arithmetic but restores quadratic state. FlashAttention instead stores a rowwise summary—the log-sum-exp statistic—and recomputes probabilities when gradients are needed [Dao et al., 2022]. The trade-off is explicit:

$$\text{more dot products and exponentials} \iff \text{far less saved activation state.} \quad (4.5)$$

On accelerators where memory traffic is the limiting resource, this can be favorable. On a small educational kernel that does not use tensor cores, the balance may be less dramatic, but the algorithmic principle is the same.

4.7 What the method does not claim

Three negative statements are as important as the positive ones.

1. FLASHATTENTION is not linear-time dense attention. Dense non-causal arithmetic remains quadratic in sequence length.
2. FLASHATTENTION is not inherently approximate. In real arithmetic it computes the same operator as materialized attention.
3. FLASHATTENTION does not guarantee a speedup for every implementation. A poor tile shape, missing tensor-core path, or insufficient occupancy can still make a specific kernel slower than a mature framework backend.

These cautions matter for an educational portfolio project. A faithful implementation of the idea may be scientifically successful even if the final benchmark shows that PyTorch SDPA remains faster on Kaggle’s GPU. The purpose of the experiments is to measure that gap honestly and explain it, not to assume the outcome in advance.

Chapter 5

CUDA Kernel Design

5.1 Mapping one attention problem to the grid

After flattening batch and head indices, each independent attention problem consists of

$$Q \in \mathbb{R}^{N_q \times D}, \quad K, V \in \mathbb{R}^{N_k \times D}. \quad (5.1)$$

The native forward kernel groups four query rows into one thread block. With a 32-thread warp size and four warps per block,

$$\text{threads per block} = 4 \times 32 = 128. \quad (5.2)$$

Warp $w \in \{0, 1, 2, 3\}$ owns query row $i = i_0 + w$ within the current query group. This ownership model gives each output row one writer and keeps synchronization simple: all four warps cooperate on the shared-memory loads, but only the owning warp updates the output state for its row.

Table 5.1: Static design parameters of the native CUDA kernels.

Parameter	Value	Purpose
Warp size	32	lane-level SIMD/SIMT execution unit
Warps per block	4	four owned rows per block
Threads per block	128	cooperative shared-memory loading and warp arithmetic
Key/value tile height B_c	16	shared-memory reuse granularity
Values per lane	8	covers $D \leq 8 \times 32 = 256$
Index type	64-bit	safe flattened addressing
Shared tile dtype	FP32	numerically stable accumulation and reuse

5.2 Per-lane data ownership

Within a row-owning warp, lane ℓ is responsible for dimensions

$$\ell, \ell + 32, \ell + 64, \dots \quad (5.3)$$

until the head dimension D is exhausted. Since the project supports $D \leq 256$, each lane owns at most eight elements of the query row, output numerator, or gradient accumulator. This lane-strided layout has two practical advantages:

1. it handles arbitrary tails without special vector-width cases;
2. it keeps the warp-wide reduction surface small, because only scalar dot products need to be reduced across lanes.

The cost is that the kernel does not use vectorized half loads or tensor-core matrix instructions. The design therefore prioritizes readability and correctness over peak throughput.

5.3 Forward shared-memory staging

Each block allocates two dynamic shared arrays:

$$\text{shared_k}[B_c \times D], \quad \text{shared_v}[B_c \times D]. \quad (5.4)$$

All threads participate in loading both arrays from global memory. The loads are naturally contiguous along the head-dimension axis, so cooperative access can be coalesced when the underlying tensor is contiguous, which the C++ binding enforces. After the load, the block executes `__syncthreads()` so that all four warps may safely read the tile.

The sequence of events inside one key tile is therefore:

1. cooperative global-memory load of K and V into shared memory;
2. block barrier;
3. warp-local score computation, online update, and value accumulation;
4. block barrier before shared-memory overwrite.

Inactive tail warps must still participate in both barriers. Returning early would risk a block-wide deadlock, because another warp could still depend on the staged tile [NVI, 2026c].

5.4 Warp reductions and scalar state

For one key row j , each lane computes a partial dot product between its local query elements and the corresponding shared-memory key elements. A sequence of warp shuffles reduces those partial sums to the full scalar score

$$s_{ij} = \alpha q_i k_j^\top. \quad (5.5)$$

The implementation uses a classic tree reduction over offsets 16, 8, 4, 2, and 1. Warp shuffles avoid allocating shared memory for the reduction and eliminate a block-wide barrier for each score [NVI, 2026c].

Only lane zero updates the scalar online-softmax state (m_i, ℓ_i) . It computes the old and new rescaling weights and broadcasts them to the rest of the warp. Every lane then rescales its own slice of the vector numerator. This split is efficient because the scalar state would otherwise be redundantly recomputed by every lane.

5.5 Causal control flow

For causal attention, query row i may use only keys $j \leq i$. The kernel expresses this condition as a cheap branch:

$$\text{if causal and } j > i, \text{ skip the score.} \quad (5.6)$$

This decision has two benefits. First, it avoids explicitly forming masked scores with value $-\infty$. Second, it removes unnecessary exponentials and value updates. The branch is uniform for all lanes in the warp because the owned query row index is shared across the warp.

The implementation deliberately restricts causal mode to $N_q = N_k$. That choice keeps the semantics unambiguous and matches the common self-attention interpretation. Non-causal mode supports rectangular cross-attention and therefore exercises distinct query and key loop bounds.

5.6 Backward launch partition

Backward propagation uses two main kernels after a scalar prepass:

1. a delta kernel computes $\delta_i = dO_i \cdot O_i$ for every query row;
2. a query-major kernel computes dQ ;
3. a key-major kernel computes dK and dV .

The key design choice is ownership. In the query-major kernel, one warp owns one dQ_i row and streams over all key/value tiles. In the key-major kernel, one warp owns one key row j and streams over query and gradient-output tiles. This partitioning removes the need for global atomic additions in the gradient outputs. Atomic-free accumulation is especially valuable for an educational implementation because it makes the result less dependent on execution order and easier to reason about.

5.7 Occupancy and resource trade-offs

No kernel parameter exists in isolation. Shared-memory footprint, register count, occupancy, and instruction throughput interact [Volkov, 2010]. Increasing the tile height B_c would increase data reuse but also raise shared-memory demand. Increasing the number of owned rows per block would improve tile reuse further, but it would also add more row-local state and therefore more registers per thread.

This repository fixes a conservative point in that design space:

- small enough to keep the code auditable;
- large enough to demonstrate cooperative K/V reuse;
- numerically stable through FP32 shared tiles and accumulators;
- not tuned for each GPU generation, head dimension, or dtype.

The resulting performance should therefore be interpreted as that of an intentionally readable kernel, not as a search over the best tile schedule that the hardware could support.

5.8 Host boundary, streams, and safety

The C++ binding checks all native preconditions with `TORCH_CHECK`: rank, contiguity, layout, dtype, device agreement, non-empty dimensions, head-dimension limits, and causal shape constraints. This boundary matters because undefined or silently coerced inputs can make low-level CUDA behavior extremely hard to diagnose.

The launcher uses PyTorch’s active device and current stream rather than assuming the CUDA default stream [PyT, 2026a]. Stream correctness is not cosmetic. A kernel that accidentally launches on the wrong stream may appear correct in isolation and fail only when composed with a larger asynchronous workload. The test suite therefore includes a non-default-stream case as a direct regression guard.

5.9 Implication for profiling

Because the kernel uses scalar arithmetic and explicit synchronization, several profiler outcomes are plausible before measurements are run:

1. reduced allocator memory relative to materialized attention;
2. lower HBM traffic for score/probability intermediates;
3. possible occupancy limits from registers or shared memory;
4. arithmetic bottlenecks compared with tensor-core SDPA backends;
5. branch and synchronization overhead that grows with small problem sizes.

The benchmark and report template are organized to test these explanations rather than merely to report elapsed time. Nsight Compute and Compute Sanitizer are therefore part of the reproducibility plan even though their artifacts are not yet present in this authoring environment [NVI, 2026b,d].

Chapter 6

Repository Implementation and Software Interface

6.1 Why software structure matters

A CUDA kernel alone is not a usable portfolio project. To evaluate and trust the operator, the repository also needs a public Python API, explicit validation behavior, benchmark tooling, packaging that behaves correctly on Kaggle, and documentation that distinguishes observed evidence from future work. This chapter documents those pieces because they determine whether the kernel can be reproduced and critiqued by another reader.

6.2 Layered repository design

The project is organized into four software layers.

1. **Reference layer.** The module `flash_attention_cuda/reference.py` implements an explicit PyTorch oracle and a PyTorch SDPA wrapper. These functions define the correctness contract independently of the native code.
2. **Autograd/dispatch layer.** The module `flash_attention_cuda/attention.py` validates inputs, chooses the requested implementation, and wraps the native extension in a custom `torch.autograd.Function`.
3. **Native boundary layer.** The file `src/attention_cuda.cpp` exposes the C++ binding, performs native input checks, and launches CUDA code through PyTorch’s extension interface [PyT, 2026b].
4. **Kernel layer.** The file `src/attention_kernel.cu` contains the forward, delta, dQ , and combined dK/dV kernels.

This separation is intentional. It allows the project to remain usable even on a machine without `nvcc`: the Python oracles and most tests still work, while the native path fails with an explicit message rather than a vague import error.

6.3 Public Python API

The public entry point is conceptually

Listing 6.1: Public API shape.

```
flash_attention(  
    q, k, v,
```

```
causal=False,
softmax_scale=None,
implementation="auto",
)
```

The wrapper serves three roles.

Input validation. The same high-level contract applies to all implementations: 4D tensors in $[B, H, N, D]$ order, shared device and floating dtype, matched batch/head dimensions, equal K/V sequence length, equal head dimension, and non-zero sequence/head sizes. Causal mode additionally requires $N_q = N_k$.

Dispatch policy. The string `implementation` selects among four behaviors:

- "reference" uses the explicit quadratic PyTorch oracle;
- "sdpa" calls `torch.nn.functional.scaled_dot_product_attention`;
- "cuda" requires the compiled extension and the native kernel contract;
- "auto" uses the native kernel when possible and falls back to SDPA otherwise.

This design is pedagogically useful because it exposes both a slow, transparent oracle and a realistic framework baseline through one stable interface.

Autograd integration. The custom `torch.autograd.Function` stores the forward inputs, output, and row log-sum-exp tensor and invokes the native backward when gradients are requested. The backward is marked once-differentiable because the repository does not claim second-order gradient support.

6.4 Reference implementations

The explicit oracle materializes the score matrix and probability matrix using ordinary PyTorch operations:

$$S = \alpha QK^\top, \quad P = \text{softmax}(S), \quad O = PV. \quad (6.1)$$

For FP16 and BF16 inputs, the oracle accumulates in FP32 before casting back to the input dtype. This makes it a stronger numerical baseline for the custom kernel than a naive same-dtype implementation would be. The SDPA wrapper uses PyTorch’s built-in attention operator with the same logical contract, which provides a modern framework comparison point [PyT, 2026d].

6.5 Native binding contract

The C++ extension is the narrowest trusted interface to the CUDA code. It performs checks that are specific to the native implementation:

- all tensors must be CUDA tensors;
- all tensors must be contiguous and strided;
- dtype must be FP16, BF16, or FP32;
- $1 \leq D \leq 256$;
- saved output and LSE tensors in backward must match the expected shapes;
- the scale must be finite and representable in FP32.

The point of these checks is not merely user convenience. They establish assumptions that the CUDA kernels depend on for correct indexing and memory access. Catching a violation in C++ with a readable error is dramatically safer than letting it reach device code.

6.6 Packaging and Kaggle build behavior

The project uses a standard PyTorch extension build via `setuptools`. The key packaging decision is that Kaggle builds should run with

Listing 6.2: Kaggle build command.

```
MAX_JOBS=2 python -m pip install -v -e . --no-build-isolation
```

so that the extension compiles against Kaggle’s already-installed PyTorch and CUDA stack rather than pulling a mismatched wheel set. The build script also supports a reference-only CPU path through the environment variable

```
FLASH_ATTENTION_SKIP_CUDA_BUILD=1
```

which disables native compilation entirely. That feature is useful for documentation work and CPU-side correctness tests in environments like the current authoring workspace.

6.7 Benchmark and artifact pipeline

The benchmark CLI in `benchmarks/bench_attention.py` measures forward or forward-backward execution across a grid of shapes, dtypes, methods, and causal modes. It records:

- per-repeat latency samples and percentile summaries;
- query-token throughput and a dense-equivalent TFLOP/s estimate;
- incremental peak allocator memory on CUDA;
- numerical error against the explicit oracle, when enabled;
- environment metadata including command line, timestamps, git state, software versions, and GPU properties.

The repository also includes `benchmarks/render_report_artifacts.py`, which converts one benchmark JSON file into optional LaTeX snippets under `report/generated/`. This keeps the long-form report tied to a machine-readable artifact rather than manual copy-paste.

6.8 Local continuity files

Two local-only files, `context.md` and `how__to_run.md`, are intentionally ignored by Git. They exist to help a single working checkout remember the intended Kaggle workflow and report-generation steps without turning that ephemeral guidance into the public repository narrative. The public source of truth remains the README, the benchmark code, and the report.

6.9 Why the interface is part of the technical contribution

For a systems portfolio project, software boundary design is part of the contribution rather than an afterthought.

1. It makes unsupported cases explicit instead of silently falling back in ways that could hide evaluation mistakes.

2. It allows CPU-only reasoning and documentation work to continue even when a GPU is unavailable.
3. It separates correctness oracles from the code under test, which prevents circular validation.
4. It turns future benchmark numbers into reproducible artifacts that can be checked independently of the prose.

This chapter therefore complements the kernel design: the CUDA code explains how the operator works, while the software structure explains how one can build, invoke, verify, and report it responsibly.

Chapter 7

Backward Pass by Recomputation

7.1 Why the backward pass is the hard part

Forward exact attention can be streamed with the online state (m_i, ℓ_i, A_i) . Backward propagation is more subtle because the derivative of softmax couples all keys in a row. A naive implementation would save the full probability matrix P or the full score matrix S so that reverse mode can reuse them later. Doing so would immediately reintroduce the quadratic memory cost that the forward pass was designed to avoid.

FlashAttention’s solution is to save only the row log-sum-exp statistic L_i and recompute probabilities on demand [Dao et al., 2022]. This section derives the required gradients and explains how the repository maps them onto two atomics-free CUDA kernels.

7.2 Reverse-mode equations

Let $G = \partial\mathcal{L}/\partial O$ be the upstream derivative of a scalar loss with respect to the attention output. For one row i ,

$$O_i = \sum_j p_{ij} v_j, \quad p_{ij} = e^{s_{ij} - L_i}. \quad (7.1)$$

The derivative with respect to the probabilities is

$$\frac{\partial\mathcal{L}}{\partial p_{ij}} = G_i v_j^\top. \quad (7.2)$$

The softmax Jacobian yields

$$\frac{\partial\mathcal{L}}{\partial s_{ij}} = p_{ij} \left(\frac{\partial\mathcal{L}}{\partial p_{ij}} - \sum_t p_{it} \frac{\partial\mathcal{L}}{\partial p_{it}} \right). \quad (7.3)$$

Define the row scalar

$$\delta_i = G_i O_i^\top. \quad (7.4)$$

Because $O_i = \sum_t p_{it} v_t$,

$$\delta_i = \sum_t p_{it} (G_i v_t^\top) = \sum_t p_{it} \frac{\partial\mathcal{L}}{\partial p_{it}}. \quad (7.5)$$

Hence

$$dS_{ij} := \frac{\partial\mathcal{L}}{\partial s_{ij}} = p_{ij} (G_i v_j^\top - \delta_i). \quad (7.6)$$

Once dS is known, the remaining matrix derivatives are ordinary products:

$$dQ = \alpha dSK, \quad (7.7)$$

$$dK = \alpha dS^\top Q, \quad (7.8)$$

$$dV = P^\top G. \quad (7.9)$$

7.3 Reconstructing probabilities

The forward pass saves $L_i = \log \sum_j e^{s_{ij}}$. This is enough to reconstruct each probability without storing P :

$$p_{ij} = e^{s_{ij} - L_i}. \quad (7.10)$$

The backward pass therefore reuses the original Q , K , and V tensors, recomputes the required dot products, and evaluates (7.6) on the fly. The extra arithmetic is the explicit price paid to avoid quadratic saved state.

7.4 Delta prepass

Equation (7.4) suggests a small preparatory kernel. One warp owns one query row and reduces

$$\delta_i = \sum_{r=0}^{D-1} G_{ir} O_{ir}. \quad (7.11)$$

The output is a compact FP32 tensor of shape $[B, H, N_q]$. This tensor is cheap to store and is reused by both the dQ kernel and the combined dK/dV kernel.

The prepass is worth separating because it simplifies the two main gradient kernels. Each main kernel can then treat δ_i as already available rather than repeatedly contracting G_i with O_i for every tile.

7.5 Query-major kernel for dQ

The dQ kernel mirrors the forward ownership pattern. Each warp owns one query-gradient row and streams over key/value tiles. For a valid pair (i, j) it computes:

$$p_{ij} = e^{\alpha q_i k_j^\top - L_i}, \quad (7.12)$$

$$dP_{ij} = G_i v_j^\top, \quad (7.13)$$

$$dS_{ij} = p_{ij}(dP_{ij} - \delta_i), \quad (7.14)$$

$$dQ_i += \alpha dS_{ij} k_j. \quad (7.15)$$

This mapping has an important structural property: one warp is the only writer of row dQ_i . Therefore no atomic update is required.

7.6 Key-major kernel for dK and dV

The remaining gradients are more naturally grouped by key row. If the repository attempted to compute dK and dV inside the query-major kernel, many query warps would need to contribute to the same key-gradient row, forcing atomics or an additional reduction buffer.

Instead, the code launches a second kernel in which each warp owns one key row j and streams through tiles of query rows and upstream gradients.

For each valid pair (i, j) the owning key warp evaluates

$$p_{ij} = e^{\alpha q_i k_j^\top - L_i}, \quad (7.16)$$

$$dP_{ij} = G_i v_j^\top, \quad (7.17)$$

$$dS_{ij} = p_{ij}(dP_{ij} - \delta_i), \quad (7.18)$$

$$dK_j += \alpha dS_{ij} q_i, \quad (7.19)$$

$$dV_j += p_{ij} G_i. \quad (7.20)$$

Again the ownership rule ensures a single writer for each output row.

Proposition 7.1 (Single-writer gradient partition). *In the repository’s backward design, every element of dQ , dK , and dV is written by exactly one warp during its owning kernel.*

Proof. The dQ kernel assigns each query row i to exactly one warp in exactly one block-group traversal, and only that warp updates row dQ_i . The key-major kernel assigns each key row j to exactly one warp in exactly one block-group traversal, and only that warp updates rows dK_j and dV_j . Since the three outputs are partitioned by row and never shared across kernels, no output element has multiple writers. $\square$

7.7 Causal handling in backward

The causal mask applies to backward exactly where it applies to forward: only pairs with $j \leq i$ contribute. The kernels implement this by skipping invalid query-key pairs during recomputation. This preserves exactness because masked probabilities are identically zero and therefore contribute nothing to (7.6) or (7.9).

7.8 Cost model of recomputation

Recomputation changes the practical runtime profile. The backward pass performs extra dot products because each relevant score is effectively revisited in both the dQ and key-major kernels. In dense non-causal attention the usual forward-backward FLOP estimate is

$$F_{\text{fwd+bwd}} \approx 12BHN_q N_k D, \quad (7.21)$$

three times the simplified forward estimate. This figure remains an approximation because it excludes exponentials, comparisons, reductions, casts, and index arithmetic. It nevertheless captures the main idea: backward is more computationally intense than forward, but it avoids a much larger activation footprint.

7.9 Numerical considerations

Backward shares the same floating-point themes as forward:

- probabilities are reconstructed from a difference of two potentially large quantities, $\alpha q_i k_j^\top$ and L_i ;
- the row contraction δ_i is a reduction and therefore order sensitive;
- dQ , dK , and dV each accumulate many contributions.

The repository keeps dot products, reconstructed probabilities, delta values, and gradient accumulators in FP32 and casts only at the final store. This does not remove rounding error, but it keeps the numerical behavior aligned with the project’s educational and validation goals.

7.10 Why the backward design is portfolio-worthy

Many simplified attention projects stop at a forward kernel because the backward path is where memory efficiency, numerical reasoning, and parallel ownership interact most sharply. This repository instead implements the complete first-order story:

1. the forward pass saves only the compact state needed for backward;
2. backward reconstructs probabilities rather than storing them;
3. the gradient outputs are partitioned to avoid atomics;
4. the public autograd wrapper exposes the operator as a trainable PyTorch function.

For a research or systems portfolio, that end-to-end differentiable path matters. It demonstrates not only that the algorithm can be derived, but that it can be integrated into the software conventions of modern deep-learning tooling.

Chapter 8

Correctness and Validation Strategy

8.1 Validation goals

For a custom attention kernel, correctness is not one binary statement. At minimum, three separate claims must be checked:

1. the Python API enforces the documented contract and dispatches predictably;
2. the native forward outputs agree with a trusted reference within justified tolerances;
3. the native first-order gradients with respect to Q, K, and V agree with a trusted reference within justified tolerances.

The repository’s test suite is organized around these claims rather than around file boundaries.

8.2 Reference choice

The explicit reference implementation is intentionally quadratic. It materializes scores and probabilities, accumulates in FP32 for half-precision inputs, and uses ordinary PyTorch operators whose semantics are easy to inspect. This makes it a better pedagogical oracle than comparing the custom kernel only against another fused kernel. The suite also compares the explicit oracle to PyTorch SDPA on CPU to ensure that the reference itself is not silently wrong.

8.3 Test layers

This split has a practical advantage: most of the repository remains testable on CPU-only machines, while the strict CUDA suite can act as a publication gate on Kaggle.

8.4 Shape and dtype coverage

The CUDA suite deliberately emphasizes awkward boundaries:

- sequence lengths below, at, and above the tile size of 16;
- head dimensions below, at, and above common warp-friendly values such as 32, 64, 128, and the maximum 256;
- rectangular non-causal cross-attention where $N_q \neq N_k$;
- square causal cases where the valid domain changes row by row;
- FP32, FP16, and BF16 where supported by the runtime.

Table 8.1: Automated validation layers currently implemented in the repository.

Layer	Location	Purpose
Reference/API tests	<code>tests/test_reference.py</code> , <code>tests/test_public_api.py</code>	validate input errors, cross-attention support, causal restrictions, explicit scale handling, non-contiguous Python inputs, and agreement between the explicit oracle and SDPA
CUDA output tests	<code>tests/test_cuda.py</code>	compare native forward output with the explicit oracle across dtypes, causal modes, tail shapes, and large logits
CUDA gradient tests	<code>tests/test_cuda.py</code>	compare dQ , dK , and dV against the explicit oracle for common upstream gradients
Availability gate	<code>tests/conftest.py</code>	turn missing GPU/extension prerequisites into a hard error when <code>-require-cuda</code> is set

This emphasis is intentional. Many CUDA bugs hide in tail handling, index arithmetic, and branch conditions rather than in idealized square powers-of-two shapes.

8.5 Tolerance policy

The repository does not demand bitwise identity from the native kernel. Such a demand would be inappropriate because different kernels legitimately reassociate floating-point operations, use different reduction orders, and sometimes invoke different backend instructions [Goldberg, 1991, Higham, 2002, PyT, 2026c]. Instead the tests set dtype-aware tolerances:

1. tight tolerances for FP32 forward and backward;
2. moderately looser tolerances for FP16 due to half-precision output stores and stronger rounding sensitivity;
3. looser still for BF16 because its mantissa is shorter than FP16’s.

The suite also checks that outputs and gradients remain finite on intentionally large-magnitude inputs. Finiteness is a coarse but valuable sanity condition: a numerically stable softmax should not explode into NaNs under the tested cases.

8.6 Non-contiguous and stream-sensitive behavior

The public Python wrapper promises to accept strided tensors by calling `.contiguous()` before invoking the native operator. That promise is tested explicitly. This is a good example of an interface-level property that matters even though the CUDA kernel itself never sees the original strided layout.

The suite also exercises a non-default CUDA stream. This case targets a specific class of integration bug: launching work on the wrong stream can look correct in a serial test and fail only in a larger asynchronous program. Since PyTorch programs often rely on implicit stream composition, this regression guard is important.

8.7 What the current suite does not yet automate

An honest report must also describe what is absent. The present repository does not yet automate:

- finite-difference gradient checks for the CUDA kernel;
- large randomized stress tests under Compute Sanitizer;
- long-duration randomized fuzzing across many shape combinations;
- deterministic cross-architecture checks across several GPU generations.

These omissions do not invalidate the current tests, but they limit the strength of any claim one should make from them. In particular, passing the existing suite is strong evidence of first-order functional correctness, not a proof that no race, OOB access, or architecture-specific corner case remains.

8.8 Strict publication gate

The most important operational feature is the explicit strict flag:

Listing 8.1: Strict CUDA correctness gate.

```
pytest -m cuda --require-cuda -q
```

When this flag is present, a missing extension or missing visible GPU is an immediate failure rather than a skipped test session. This prevents a common documentation mistake in GPU projects: a green test run that is green only because the native tests silently never executed.

8.9 Interpreting a passing result

If the strict CUDA suite passes on Kaggle, the following claims become justified:

1. the custom forward outputs numerically match the explicit oracle on the tested grid;
2. the custom first-order gradients numerically match the explicit oracle on the tested grid;
3. the public API contract behaves as documented for supported and unsupported cases;
4. the implementation runs on PyTorch’s current CUDA stream and on non-contiguous wrapper inputs.

Nothing more should be inferred automatically. A passing correctness suite does not establish benchmark superiority, optimality, or completeness relative to the production FlashAttention libraries.

Chapter 9

Experimental Methodology

9.1 Principles

The benchmark design follows four principles.

1. **Separate correctness from timing.** Performance numbers are meaningful only after the strict CUDA correctness suite passes.
2. **Record the environment.** GPU model, compute capability, PyTorch version, CUDA version, nvcc availability, and git state are part of the result.
3. **Preserve failures.** Out-of-memory events, unavailable methods, and validation skips are data, not noise to discard.
4. **Report distributions.** A single mean latency hides variability; the benchmark records repeated samples and percentile summaries.

9.2 Measured implementations

The benchmark can compare up to four implementations behind one public API:

1. `cuda`: the custom native extension;
2. `sdpa`: PyTorch scaled-dot-product attention;
3. `reference`: the explicit quadratic oracle;
4. `auto`: wrapper dispatch that chooses native or SDPA based on compatibility.

For a fair methodological narrative, the most important comparison is not always the most flattering one. The custom kernel should be compared both with the transparent quadratic reference and with PyTorch’s optimized fused backend. The former explains whether fusion and streaming help at all; the latter explains how far an educational kernel remains from production engineering.

9.3 Shape matrix

The benchmark treats one run as a Cartesian product over:

- query lengths;
- optional key lengths;
- head dimensions;
- dtypes;
- causal mode;
- execution mode: forward only or forward-backward.

The CLI now supports

```
--causal false
--causal true
--causal both
```

so a single artifact may cover non-causal, causal, or both cases. This is useful for a report because the causal schedule changes the valid work domain and can materially change both runtime and memory behavior.

9.4 Timing procedure

On CUDA devices the benchmark uses CUDA Events around repeated execution. This isolates device-side elapsed time more reliably than a host-side wall clock alone. For CPU execution it falls back to `time.perf_counter`. Each configuration follows the same sequence:

1. allocate and seed inputs;
2. optionally build a reference output for error metrics;
3. warm up the selected implementation;
4. collect repeated timing samples;
5. measure incremental peak memory separately;
6. serialize both per-row results and environment metadata.

Warmup matters because the first call may trigger lazy initialization, kernel loading, or cache effects that are not representative of steady-state operation.

9.5 Reported metrics

Table 9.1: Primary benchmark outputs written per successful row.

Metric	Meaning
<code>latency_ms_p20, median, p80, p95</code>	latency distribution summary over repeated runs
<code>query_tokens_per_second</code>	BN_q /seconds, intentionally not multiplied by head count
<code>dense_equivalent_tflops</code>	simplified FLOP-rate estimate using the dense forward or forward-backward model
<code>incremental_peak_memory_bytes</code>	delta or peak during one operation minus baseline allocation with resident inputs
<code>max_abs_error,</code> <code>mean_abs_error,</code> <code>max_relative_error</code>	numerical deviation from the explicit oracle, when validation is enabled
<code>reference_status</code>	whether the explicit oracle was run, skipped for size, or failed

The FLOP-rate figure is explicitly labeled “dense-equivalent” because it is a model, not a hardware instruction count. It excludes softmax scalar work, exponentials, control flow, and address arithmetic. Its purpose is comparative interpretation, not hardware billing.

9.6 Environment capture

Every benchmark JSON also stores a metadata block that includes:

- timestamp and full CLI invocation;
- repository path, git commit, dirty-state marker, and status output;
- Python, PyTorch, and CUDA version strings;
- `nvcc` location and version when available;
- CUDA device names, compute capabilities, and memory sizes;
- selected environment variables such as `CUDA_VISIBLE_DEVICES`.

This capture is critical for Kaggle, where the notebook image may change over time and where reinstalling PyTorch is often counterproductive.

9.7 Kaggle protocol

The intended GPU validation environment is a Kaggle notebook with GPU acceleration enabled. The procedural rules are simple but important:

1. work in `/kaggle/working`, not read-only `/kaggle/input`;
2. inspect the preinstalled PyTorch and CUDA environment before compiling;
3. build against Kaggle’s stack with `-no-build-isolation`;
4. run CPU/API tests first, then the strict CUDA suite;
5. preserve the benchmark CSV and JSON artifact before editing the report.

Reproducibility requirement

The authoring environment used for this report does not provide a CUDA-capable GPU or `nvcc`. Therefore this chapter defines the methodology but intentionally does not claim any measured native results.

9.8 From benchmark artifact to report

Manual transcription of benchmark numbers into a report is error-prone. The repository therefore includes a small renderer:

Listing 9.1: Generating report snippets from one benchmark JSON artifact.

```
python -m benchmarks.render_report_artifacts \
--benchmark-json results/runs/benchmark_<run-id>.json \
--output-dir report/generated
```

The renderer emits optional LaTeX files for environment, correctness, performance, memory, and profiling summaries. The report uses them only if they exist. Otherwise it displays explicit placeholders.

9.9 What would constitute convincing evidence

The report should treat the following bundle as the minimum credible evidence package for a public benchmark claim:

1. passing strict CUDA correctness test log;
2. benchmark CSV and JSON;
3. compiler/build log;
4. environment summary identifying the GPU and software stack;
5. profiler or sanitizer artifacts for any microarchitectural claim beyond raw timing.

This evidentiary standard is stricter than many small repositories apply, but it is appropriate for a portfolio piece that aims to be both technically serious and honest about what has and has not yet been measured.

Chapter 10

Results, Placeholders, and Future Evidence

10.1 Status of empirical evidence

This repository is intentionally prepared to host rigorous results without fabricating them. At the time of writing, the authoring environment does not expose the GPU toolchain required to build and run the native extension. Consequently, the report treats all native empirical claims as pending until benchmark artifacts are generated on Kaggle or another recorded CUDA machine.

RESULT_PENDING: No native benchmark artifact in the authoring environment

No GPU build, strict CUDA test log, or benchmark JSON produced by the custom CUDA path is available in this checkout at report-authoring time. All native performance, memory, and profiler sections below are therefore templates awaiting a real artifact.

10.2 Environment summary

When a benchmark JSON is rendered into `report/generated/environment.tex`, it should identify the exact hardware and software used to obtain the numbers in this chapter.

RESULT_PENDING: Environment summary pending

Generate `generated/environment.tex` from the chosen benchmark JSON with `python -m benchmarks.render_report_artifacts`. Do not replace this box by hand.

10.3 Correctness summary

The first empirical question is not speed but agreement with the reference. The rendered correctness summary should report the largest observed forward discrepancies across successful benchmark rows and should be interpreted together with the stricter unit-test suite.

RESULT_PENDING: Correctness summary pending

Populate `generated/correctness_summary.tex` only from a benchmark JSON whose custom CUDA rows were produced after the strict CUDA test suite passed.

10.4 Performance comparison

The performance table should compare the custom CUDA kernel against both PyTorch SDPA and the explicit reference when those baselines are available for the same shapes. The goal is interpretation, not marketing:

1. speedup over the reference indicates whether fusion and reduced IO help at all;
2. comparison with SDPA indicates how far the educational kernel remains from production engineering;
3. shape-by-shape variation indicates whether launch overhead or tile reuse dominates.

RESULT_PENDING: Performance summary pending

No rendered performance table is present. After a benchmark run, generate this file from the JSON artifact instead of copying values manually into the report.

10.5 Memory behavior

Because the project’s central algorithmic claim is about avoiding quadratic intermediates, memory evidence is as important as latency evidence. The rendered memory table should therefore be read together with the benchmark methodology that defines “incremental peak memory” as the allocator peak beyond already-resident inputs.

RESULT_PENDING: Memory summary pending

No rendered memory table is present. The eventual artifact should compare methods on the same shape grid and retain unsuccessful or OOM rows elsewhere in the CSV/JSON record.

10.6 Profiler and sanitizer evidence

Elapsed time alone does not explain why one kernel is faster or slower. Any claim about occupancy, memory bandwidth, branch divergence, cache behavior, or synchronization overhead should be backed by profiler or sanitizer artifacts.

RESULT_PENDING: Profiling summary pending

No profiler summary has been generated. Before drawing microarchitectural conclusions, attach Nsight Compute and Compute Sanitizer evidence and summarize it in a derived `generated/profiling_summary.tex` file.

10.7 Publication rules for future updates

When this chapter is updated with real results, the following rules should remain in force:

1. every table must trace back to a stored benchmark artifact;
2. benchmark commands and environment metadata must be preserved;
3. failing or unavailable configurations must not be silently removed from the raw artifact;
4. the README and this report must be updated together if they make public empirical claims;
5. the narrative must distinguish measured observations from causal explanations that still require profiling support.

This policy is part of the portfolio value of the repository. It shows not only that the code exists, but that the project is prepared to support reproducible, falsifiable claims once the Kaggle run is complete.

Chapter 11

Discussion

11.1 What this project demonstrates already

Even before benchmark results are filled in, the repository demonstrates several nontrivial technical outcomes.

1. Exact attention can be implemented without materializing the score or probability matrix.
2. The online-normalization recurrence can be carried all the way into a working CUDA kernel with saved LSE statistics.
3. The backward pass can be expressed through recomputation and single-writer ownership rather than through quadratic saved state or atomics.
4. A small PyTorch extension can expose this behavior behind a clean, tested Python API.

Those are substantive systems outcomes independent of whether the final Kaggle benchmark beats PyTorch SDPA.

11.2 Why an educational kernel may still lose to production code

If future results show that the custom kernel is slower than PyTorch SDPA or the production FlashAttention libraries, that would not be surprising. The production stack benefits from optimizations deliberately omitted here:

- tensor-core matrix instructions and architecture-specific scheduling;
- more aggressive tiling and work partitioning [Dao, 2024];
- asynchronous pipelines and low-precision techniques [Shah et al., 2024];
- broader kernel specialization by head dimension, dtype, and mask type;
- substantial tuning effort across GPU generations.

The educational kernel instead spends resources on inspectability: FP32 shared tiles, scalar arithmetic, fixed tile sizes, and direct expression of the online invariant.

11.3 What a positive result would mean

Suppose a future benchmark shows that the custom kernel outperforms the explicit reference for moderate or long sequences while still lagging SDPA. That would support a focused conclusion:

The repository successfully demonstrates the algorithmic value of IO-aware fusion and recomputation, while also illustrating the performance gap between a readable teaching implementation and a highly optimized production kernel.

This is a strong portfolio outcome because it connects theory, implementation, and honest measurement.

11.4 What a negative result would still teach

Suppose instead that the custom kernel fails to outperform even the explicit materialized baseline on Kaggle. That would still be informative. It would suggest that one or more of the following dominates on the tested hardware:

1. launch overhead for small or moderate sequence lengths;
2. insufficient occupancy due to registers or shared memory;
3. scalar arithmetic that cannot exploit tensor-core throughput;
4. extra backward recomputation that outweighs the saved memory traffic for the tested shapes;
5. benchmark baselines that already use highly optimized fused paths.

Such a result would motivate targeted profiling and help explain why production FlashAttention evolved toward more specialized designs.

11.5 Relationship to the broader literature

The repository sits at an intersection of three threads in the literature:

1. the Transformer attention formulation of [Vaswani et al. \[2017\]](#);
2. memory-efficient exact attention ideas such as [Rabe and Staats \[2021\]](#);
3. IO-aware GPU scheduling and modern FlashAttention kernels [[Dao et al., 2022](#), [Dao, 2024](#), [Shah et al., 2024](#)].

Its contribution is not a new asymptotic result. It is a compact end-to-end realization that makes the central ideas inspectable and reproducible in a single repository.

11.6 Portfolio perspective

From a hiring or research-portfolio perspective, the most impressive part of a project like this is rarely a raw TFLOP/s number in isolation. What stands out is the ability to:

- derive the math carefully;
- map it into CUDA execution and memory scopes;
- integrate it with a real deep-learning runtime;
- test it rigorously;
- document limitations and evidence policy honestly.

That is the standard this report is designed to support.

Chapter 12

Limitations and Honest Boundaries

12.1 Native feature limits

Table 12.1: Current limitations of the native implementation and why they matter.

Limitation	Status	Consequence
Head dimension cap	$D \leq 256$	keeps register and shared-memory usage simple, but excludes larger heads
No dropout	unsupported	not a drop-in training replacement for general Transformer workloads
No arbitrary mask or bias	unsupported	no padding masks, additive biases, ALiBi, or custom sparsity patterns
No GQA/MQA	unsupported	Q, K, and V must have the same head count
No second-order gradients	unsupported	backward is sufficient for ordinary training but not higher-order differentiation
No tensor-core path	omitted by design	likely leaves substantial performance on the table
Fixed tile sizes	fixed at compile time	easy to analyze, but not tuned per architecture or workload
Authoring environment has no GPU	currently true	empirical claims must wait for Kaggle or another CUDA machine

12.2 Validation limits

The present tests are strong enough to support first-order correctness claims on the exercised grid, but they are not exhaustive. In particular:

- a passing suite does not rule out all race conditions or OOB accesses;
- the suite does not yet cover every possible GPU architecture or driver stack;
- numerical tolerance checks are necessary but cannot prove ideal conditioning;
- benchmark-based memory measurements rely on allocator statistics rather than a complete device-memory trace.

12.3 Performance interpretation limits

Any future performance discussion in this repository must respect three caveats.

First, a dense-equivalent TFLOP/s estimate is not a true instruction-throughput metric. It is a convenient normalization based on the mathematical work model.

Second, benchmark results on Kaggle’s GPU should not be generalized automatically to other architectures. Shared-memory capacity, tensor-core behavior, cache geometry, and compiler quality all matter.

Third, comparison with PyTorch SDPA is comparison with a moving target. PyTorch may route to math, efficient, cuDNN, or FlashAttention-like backends depending on version, dtype, mask, and hardware [PyT, 2026d]. That is why the benchmark records backend-related metadata and allows explicit SDPA-backend forcing.

12.4 Scientific scope

This project should be read as a serious implementation study, not as a claim to state-of-the-art attention performance. Its main scientific scope is narrower and more defensible:

1. demonstrate the exact online-softmax algorithm in code;
2. implement the corresponding first-order backward pass;
3. expose the operator through a modern PyTorch extension interface;
4. define a reproducible benchmark and reporting workflow.

12.5 Future work

Natural next steps are clear:

- tune tile sizes by head dimension and architecture;
- add vectorized loads and tensor-core kernels;
- support dropout, masks, and head-count asymmetry;
- add profiler-driven optimization and sanitization passes;
- automate more report generation directly from benchmark and profiler artifacts.

The value of the current repository is that those directions are now grounded in a complete, understandable baseline rather than in an unexplained black box.

Chapter 13

Conclusion

This report has developed an educational FlashAttention-style CUDA operator from first principles. The project begins with the exact scaled dot-product attention equation, rewrites it through an online softmax invariant, maps that invariant onto a tiled CUDA schedule, and extends the same design into a first-order backward pass based on recomputation rather than quadratic saved state.

The resulting repository is more than a kernel demo. It exposes a usable PyTorch interface, explicit native-contract checks, correctness oracles, a strict CUDA test gate, a benchmark harness with environment capture, and a LaTeX report template that refuses to invent results. That combination is what makes the project portfolio-ready: the code, the math, the tests, and the reporting discipline all reinforce each other.

Future Kaggle runs will determine the empirical standing of the implementation relative to a transparent quadratic baseline and to PyTorch’s optimized SDPA path. Whatever those numbers turn out to be, the repository is now structured to interpret them honestly. If the custom kernel wins on some shapes, the result will illustrate the power of IO-aware fusion. If it loses, the gap will still illuminate why production attention kernels require the deeper optimization techniques explored by FlashAttention-2 and FlashAttention-3.

Either way, the core lesson remains: efficient GPU programming is not only about counting arithmetic operations. It is about choosing what to store, what to recompute, and where each piece of data should live while the computation unfolds. FlashAttention makes that lesson unusually concrete, and this repository turns it into an inspectable end-to-end implementation.

Appendix A

Additional Derivations

A.1 Merging two processed key sets

Let J_1 and J_2 be disjoint processed key sets for one query row. Define

$$m_a = \max_{j \in J_a} s_{ij}, \quad (\text{A.1})$$

$$\ell_a = \sum_{j \in J_a} e^{s_{ij} - m_a}, \quad (\text{A.2})$$

$$A_a = \sum_{j \in J_a} e^{s_{ij} - m_a} v_j, \quad (\text{A.3})$$

for $a \in \{1, 2\}$. Let $m = \max(m_1, m_2)$. Then the merged state is

$$\ell = e^{m_1 - m} \ell_1 + e^{m_2 - m} \ell_2, \quad (\text{A.4})$$

$$A = e^{m_1 - m} A_1 + e^{m_2 - m} A_2. \quad (\text{A.5})$$

This follows from the same change-of-reference argument used in the online recurrence. It shows that tiles may be processed independently and merged later so long as their state uses the same sufficient statistics.

A.2 Softmax Jacobian contraction

For one row $p = \text{softmax}(s)$, the Jacobian is

$$\frac{\partial p_j}{\partial s_t} = p_j(\delta_{jt} - p_t), \quad (\text{A.6})$$

where δ_{jt} is the Kronecker delta. Let $g_j = \partial \mathcal{L} / \partial p_j$. Then

$$\frac{\partial \mathcal{L}}{\partial s_t} = \sum_j g_j \frac{\partial p_j}{\partial s_t} \quad (\text{A.7})$$

$$= \sum_j g_j p_j (\delta_{jt} - p_t) \quad (\text{A.8})$$

$$= g_t p_t - p_t \sum_j g_j p_j \quad (\text{A.9})$$

$$= p_t \left(g_t - \sum_j g_j p_j \right). \quad (\text{A.10})$$

In attention, $g_t = G_i v_t^\top$ and the weighted sum becomes $G_i O_i^\top$, which is exactly the scalar δ_i used by the implementation.

A.3 Causal work count

For square causal attention with sequence length N , the number of valid (i, j) pairs is

$$\sum_{i=1}^N i = \frac{N(N+1)}{2}. \quad (\text{A.11})$$

Relative to the dense non-causal case of N^2 pairs, the leading ratio tends to one half as N grows:

$$\frac{N(N+1)/2}{N^2} = \frac{1}{2} + \frac{1}{2N}. \quad (\text{A.12})$$

Finite-size tile boundaries and control flow prevent real runtimes from following this ratio exactly, but it remains a useful leading-order intuition.

A.4 Shared-memory footprint

The forward and gradient kernels stage two FP32 tiles of height $B_c = 16$ and width D :

$$2 \times 16 \times D \times 4 \text{ bytes} = 128D \text{ bytes}. \quad (\text{A.13})$$

At $D = 64, 128$, and 256 , this gives 8 KiB, 16 KiB, and 32 KiB respectively. This linear dependence on D is one reason the current native contract stops at $D = 256$.

A.5 Dense-equivalent FLOP model

The report uses the common simplified count that treats one multiply-add as two FLOPs.

Forward. The score product $QK^\top$ costs approximately $2N_qN_kD$ FLOPs and the value product PV costs another $2N_qN_kD$, giving

$$F_{\text{fwd}} \approx 4BHN_qN_kD. \quad (\text{A.14})$$

Backward. The gradient products for dQ , dK , and dV contribute the same leading order again, and recomputation adds another score-like pass. The benchmark therefore uses

$$F_{\text{fwd+bwd}} \approx 12BHN_qN_kD \quad (\text{A.15})$$

as a dense-equivalent estimate. This is a deliberately simple model rather than a complete instruction count.

Appendix B

Reproducibility Checklist

B.1 Kaggle execution checklist

1. Enable a GPU accelerator in the Kaggle notebook settings.
2. Clone or copy the repository into `/kaggle/working`.
3. Inspect the preinstalled environment:

```
python - <<'PY'
import torch
from torch.utils.cpp_extension import CUDA_HOME
print("torch:", torch.__version__)
print("torch CUDA:", torch.version.cuda)
print("CUDA available:", torch.cuda.is_available())
print("CUDA_HOME:", CUDA_HOME)
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
    print("capability:", torch.cuda.get_device_capability(0))
PY
nvcc --version
```

4. Install the repository without replacing Kaggle's PyTorch stack:

```
MAX_JOBS=2 python -m pip install -r requirements.txt
MAX_JOBS=2 python -m pip install -v -e . --no-build-isolation
```

5. Run the portable tests:

```
pytest -m "not cuda" -q
```

6. Run the strict native gate:

```
pytest -m cuda --require-cuda -q
```

7. Launch the chosen benchmark sweep and preserve the CSV/JSON artifacts.
8. Render report snippets from the chosen JSON artifact:

```
python -m benchmarks.render_report_artifacts \
    --benchmark-json results/runs/benchmark_<run-id>.json \
    --output-dir report/generated
```

9. Rebuild the report and archive the resulting PDF with the raw artifacts.

B.2 Suggested artifact bundle

For a defensible release or portfolio submission, archive the following together:

- benchmark CSV and JSON files;
- strict CUDA pytest log;
- `pip install` build log;
- PDF report and any generated report snippets;
- optional profiler output such as `.ncu-rep` or sanitizer logs.

B.3 Recommended sanitizer and profiler commands

These commands are not run in the current authoring environment, but they are the intended next diagnostic step on a CUDA machine:

Listing B.1: Examples for deeper validation on a CUDA host.

```
compute-sanitizer --tool memcheck \
  python -m pytest -m cuda --require-cuda -q

compute-sanitizer --tool racecheck \
  python -m pytest -m cuda --require-cuda -q

ncu --set full --target-processes all \
  python -m benchmarks.bench_attention \
    --device cuda --methods cuda sdpa reference \
    --seq-lens 512 1024 --head-dims 64 --dtypes float16 --causal both
```

B.4 Worktree and provenance policy

The benchmark metadata records whether the repository was dirty at run time. This does not prohibit running benchmarks on an uncommitted worktree, but it does mean that any public claim should retain the exact worktree state or commit used to generate the result. In practice, that means either committing the tested state or preserving a patch alongside the benchmark artifact.

B.5 Report update policy

When replacing placeholders in Chapter 10, follow this order:

1. generate or select the benchmark JSON artifact;
2. render LaTeX snippets from that artifact;
3. rebuild the PDF and check that the source artifact path appears in the rendered tables;
4. only then edit surrounding prose to interpret the new evidence.

This order ensures that the narrative grows out of the stored measurement rather than the other way around.

Appendix C

Benchmark and Report Artifact Schema

C.1 Benchmark JSON structure

The benchmark harness writes a JSON object with two top-level keys:

Listing C.1: Conceptual benchmark JSON schema.

```
{
  "schema_version": 1,
  "metadata": { ... },
  "results": [
    { ... one benchmark row ... },
    ...
  ]
}
```

Metadata block. The `metadata` object records run-level information such as timestamp, command line, git state, software versions, CUDA device properties, and benchmark CLI configuration.

Results block. Each entry in `results` corresponds to one method-shape-dtype-causality configuration. Rows preserve both success and failure states. This is important: a report may summarize only successful rows, but the raw artifact should retain OOMs, unavailable methods, and invalid configurations.

C.2 Representative row fields

Table C.1: Representative fields in one benchmark result row.

Field	Example type	Meaning
<code>status</code>	string	ok, error, oom, unavailable, or invalid_configuration
<code>method</code>	string	cuda, sdpa, reference, or auto
<code>dtype</code>	string	input tensor dtype name

Field	Example type	Meaning
query_length, key_length, head_dim	integers	problem shape parameters
causal	boolean	causal or non-causal mode
mode	string	forward or forward-backward
latency_ms_median	float or null	median timing sample for successful rows
dense_equivalent_tflops	float or null	simplified FLOP-rate estimate
incremental_peak_memory_mib	float or null	incremental allocator peak in MiB
max_abs_error	float or null	maximum absolute forward error versus the reference
reference_status	string	whether reference validation ran, was skipped, or failed
error	string or null	exception text for unsuccessful rows

C.3 Rendered report snippets

The helper script `benchmarks/render_report_artifacts.py` derives the following optional files from one benchmark JSON artifact:

- `report/generated/environment.tex`
- `report/generated/correctness_summary.tex`
- `report/generated/performance_summary.tex`
- `report/generated/memory_summary.tex`
- `report/generated/profiling_summary.tex`

Each generated file should:

1. state the source benchmark artifact path;
2. contain only derived information from that artifact;
3. remain safe to delete and regenerate;
4. never become the sole location of a result that is absent from the raw JSON.

C.4 Expected workflow

Listing C.2: Result-to-report workflow.

```
python -m benchmarks.bench_attention \
  --device cuda --methods cuda sdpa reference \
  --seq-lens 128 512 1024 --head-dims 64 --dtypes float16 --causal both \
  --output-dir results/runs --require-cuda

python -m benchmarks.render_report_artifacts \
  --benchmark-json results/runs/benchmark_<run-id>.json \
  --output-dir report/generated

cd report
latexmk -pdf main.tex
```

C.5 Why a schema appendix belongs in the report

This appendix may seem unusual for a course-style report, but it is valuable for a systems portfolio project. The point is to make the evidence pipeline inspectable. Another reader should be able to trace a table in Chapter 10 back to a concrete JSON artifact and then back to the exact benchmark command that produced it.

Bibliography

- Alok Aggarwal and Jeffrey Scott Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988. doi: 10.1145/48529.48535.
- Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2307.08691. URL <https://openreview.net/forum?id=mZn2Xyh9Ec>.
- Tri Dao and contributors. flash-attention: Fast and memory-efficient exact attention. GitHub repository, 2026. URL <https://github.com/Dao-AILab/flash-attention>. Reference production implementation; accessed 2026-07-17.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16363, 2022. doi: 10.48550/arXiv.2205.14135. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract.html.
- David Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991. doi: 10.1145/103162.103163.
- Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, 2 edition, 2002. doi: 10.1137/1.9780898718027.
- IEEE Standard for Floating-Point Arithmetic*. IEEE, 2019.
- Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefer. Data movement is all you need: A case study on optimizing transformers. In *Proceedings of Machine Learning and Systems*, volume 3, pages 711–732, 2021. URL <https://proceedings.mlsys.org/paper/2021/hash/bc86e95606a6392f51f95a8de106728d-Abstract.html>.
- Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax. *arXiv preprint arXiv:1805.02867*, 2018. doi: 10.48550/arXiv.1805.02867. URL <https://arxiv.org/abs/1805.02867>.
- John Nickolls, Ian Buck, Michael Garland, and Kevin Skadron. Scalable parallel programming with CUDA. *ACM Queue*, 6(2):40–53, 2008. doi: 10.1145/1365490.1365500.
- CUDA C++ Best Practices Guide*. NVIDIA Corporation, 13.3 edition, 2026a. URL <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>. Accessed 2026-07-17.
- Nsight Compute Profiling Guide*. NVIDIA Corporation, 13.3 edition, 2026b. URL <https://docs.nvidia.com/nsight-compute/ProfilingGuide/>. Accessed 2026-07-17.
- CUDA Programming Guide*. NVIDIA Corporation, 13.3 edition, 2026c. URL <https://docs.nvidia.com/cuda/cuda-programming-guide/>. Accessed 2026-07-17.

- Compute Sanitizer User Manual*. NVIDIA Corporation, 2026d. URL <https://docs.nvidia.com/compute-sanitizer/ComputeSanitizer/>. Accessed 2026-07-17.
- CUDA Semantics*. PyTorch Contributors, pytorch 2.12 edition, 2026a. URL <https://docs.pytorch.org/docs/2.12/notes/cuda.html>. Developer note; accessed 2026-07-17.
- torch.utils.cpp_extension*. PyTorch Contributors, pytorch 2.12 edition, 2026b. URL https://docs.pytorch.org/docs/2.12/cpp_extension.html. API documentation; accessed 2026-07-17.
- Numerical Accuracy*. PyTorch Contributors, pytorch 2.12 edition, 2026c. URL https://docs.pytorch.org/docs/2.12/notes/numerical_accuracy.html. Developer note; accessed 2026-07-17.
- torch.nn.functional.scaled_dot_product_attention*. PyTorch Contributors, pytorch 2.12 edition, 2026d. URL https://docs.pytorch.org/docs/2.12/generated/torch.nn.functional.scaled_dot_product_attention.html. API documentation; accessed 2026-07-17.
- Markus N. Rabe and Charles Staats. Self-attention does not need $o(n^2)$ memory. *arXiv preprint arXiv:2112.05682*, 2021. doi: 10.48550/arXiv.2112.05682. URL <https://arxiv.org/abs/2112.05682>.
- Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. *arXiv preprint arXiv:2407.08608*, 2024. doi: 10.48550/arXiv.2407.08608. URL <https://arxiv.org/abs/2407.08608>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://papers.nips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.
- Vasily Volkov. Better performance at lower occupancy. In *Proceedings of the 2010 ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–12, 2010. doi: 10.1109/SC.2010.23.
- Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.